



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра практической и прикладной информатики (ППИ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологические основы интернета вещей»

Практические работы №5-8

Группа №10

Студенты группы *ИКБО-50-23, Астахов С.
П., Враженко Д., Бурыхин И.*

(подпись)

Старший
преподаватель *Образцов Владимир Михайлович*

(подпись)

Отчет представлен «___»_____2025г.

Москва 2025 г.

Содержание

Практическая работа №5 - Измерительные и исполнительные.3	
устройства в Интернете вещей.....3	
Практическая работа №6 - Основы работы с протоколом	
MQTT.....9	
Практическая работа №7 - Форматы представления данных. .17	
Практическая работа №8 - Визуализация данных в Интернете	
вещей.....47	
Вывод о проделанных работ.....49	
Дополнительная литература.....50	

Практическая работа №5 - Измерительные и исполнительные устройства в Интернете вещей

Краткое теоретическое введение

Физические устройства Интернета вещей чаще всего состоят из нескольких типов элементов:

Датчик – чувствительный элемент, который контактирует с окружающей средой: измеряет показания, реагирует на объекты и создает сигнал об этом.

Микроконтроллер (или гейт) – специальная микросхема, предназначенная для управления различными электронными устройствами.

Типичный микроконтроллер сочетает на одном кристалле функции процессора и периферийных устройств, содержит ОЗУ и (или) ПЗУ. По сути, это однокристалльный компьютер, способный выполнять относительно простые задачи и реализовать логику работы физического устройства.

Актуатор – исполнительное устройство, например, реле или транзистор, способные что-то переключать – по сигналу от микроконтроллера или по дистанционной команде, которую примет радиомодуль. Например, если температура выйдет за допустимый предел, актуатор может включить вентиляцию или кондиционер.

Данный тип элементов Интернета вещей предназначен для воздействия на окружающую среду или определенный объект в ней.

К актуаторам относят:

- Сервоприводы
- Динамики
- Электронные замки
- Осветительные приборы

- Реле и транзисторы
- ИК-пульты

Источник питания. Для работы микроконтроллера и подключенных к нему устройств необходимо питание. В зависимости от энергопотребления и задачи мы можем питать устройство от батарейки или от сети по проводу.

Радиомодуль – устройство для организации различных технологий беспроводной связи(например, GPRS, LTE, NB-IoT, GSM, Wi-Fi, Bluetooth и др.)

Описание хода выполнения работы

WB-demo-kit v2(сервый кейс):

1. Датчик температуры 1-wire DS18B20

Параметр	Значение
Название	Цифровой термометр DS18B20
Тип измерения	Цифровой
Измеряемые параметры	Температура: -55...+125 °C
Точность	±0.5°C (в диапазоне -10...+85 °C)
Напряжение	3.0В - 5.5В
Индификатор	/dev/w1_bus_master1/28-xxxxxxxxxxxxxx
Протокол	1-Wire
Интерфейс	1-Wire
Схема подключения	WB 1-Wire схема

Пояснение: Каждый DS18B20 имеет уникальный 64-битный код, автоматически определяемый системой. Два датчика будут иметь разные адреса в формате 28-xxxxxxxxxxxxxx

2. Уровень шума в составе датчика WB-MSW v.3 (5)

Параметр	Значение
Название	Датчик звука WB-MSW v.3
Тип измерения	Аналоговый
Измеряемые параметры	Уровень звукового давления: 30-120 дБ
Точность	±1.5 дБ
Напряжение	.3В
Индификатор	/dev/i2c-1 (адрес 0x5D)
Протокол	I ² C
Интерфейс	I ² C
Схема подключения	WWB-MSW v.3

Пояснение: Микрофонный датчик измеряет звуковое давление и преобразует в цифровое значение через I²C интерфейс.

3. Модуль реле 3-канальный WB-MR3 (21)

Параметр	Значение
Название	3-канальное реле WB-MR3
Тип измерения	Исполнительное устройство
Управляемый параметр	Коммутация нагрузки: 250VAC/5A, 30VDC/5A
Точность	-
Напряжение	12-24В
Индификатор	/dev/ttyMOD4 (Modbus адрес 21)

Протокол	Modbus RTU
Интерфейс	RS-485
Схема подключения	WB-MR3

Пояснение: Управляется по Modbus RTU, каждый канал может независимо включать/выключать нагрузку.

WB-DEMO-KIT v.3(черный кейс)

1. Преобразователь для цифровых термометров WB-M1W2 (A7)

Параметр	Значение
Название	1-Wire хаб WB-M1W2
Тип измерения	Преобразователь интерфейса
Управляемый параметр	Преобразование UART ↔ 1-Wire
Точность	-
Напряжение	3.3В
Индификатор	/dev/ttyS1
Протокол	UART ↔ 1-Wire
Интерфейс	UART, 1-Wire
Схема подключения	WB-M1W2

НазначениеПреобразование UART ↔ 1-Wire

Пояснение: Позволяет подключать до 64 1-Wire устройств к одному UART порту контроллера.

2. Контроллер Wiren Board 8 (A2)

Параметр	Значение
Название	Контроллер Wiren Board 8
Тип измерения	Платформа управления
Функции	Сбор данных, управление, сетевое взаимодействие
Точность	-
Напряжение	12-24В
Индификатор	Локальный IP-адрес (напр., 192.168.1.10)
Протокол	HTTP, MQTT, Modbus TCP
Интерфейс	Ethernet, RS-485, I ² C, 1-Wire, GPIO
Схема подключения	WB8

Пояснение: Основной контроллер системы, обеспечивающий сбор данных со всех датчиков и управление исполнительными устройствами.

3. Датчик протечки (B1)

Параметр	Значение
Название	Датчик протечки воды
Тип измерения	Цифровой (дискретный)
Изменяемые параметры	Обнаружение воды: есть/нет
Точность	Двоичное состояние (сработал/не сработал)
Напряжение	3.3В (через подтяжку)
Индификатор	ExtIn1 (GPIO вход)

Протокол	GPIO (цифровой вход)
Интерфейс	GPIO
Схема подключения	Датчик протечки

Пояснение: При попадании воды на контакты замыкает цепь, что фиксируется как изменение состояния GPIO входа.

Практическая работа №6 - Основы работы с протоколом MQTT.

Краткое теоретическое введение

MQTT (англ. message queuing telemetry transport) — упрощённый сетевой протокол, работающий поверх TCP/IP, ориентированный на обмен сообщениями между устройствами по принципу издатель-подписчик.

Протокол ориентируется на простоту в использовании, невысокую нагрузку на каналы связи, работу в условиях постоянной потери связи, лёгкую встраиваемость в любую систему. Основное предназначение — работа с телеметрией от различных датчиков и устройств. Использование шаблона подписчика обеспечивает возможность устройствам выходить на связь и публиковать сообщения, которые не были заранее известны или предопределены, в частности, протокол не вводит ограничений на формат передаваемых данных.

Главные особенности протокола MQTT:

Компактный и легковесный — минимальные накладные расходы на пересылку данных, для экономии трафика.

Устойчивость к потерям — гарантированная доставка в условиях нестабильных сетевых подключений.

Асинхронный — позволяет обслуживать большое количество устройств, и не зависит от сетевых задержек.

Поддержка QoS — возможность управлять приоритетом сообщений и гарантировать доставку сообщения адресату.

Динамическая конфигурация — не требует предварительно согласования полей и форматов данных, может конфигурироваться «на лету».

Работает за NAT — клиенты могут находиться за NAT, только сервер (брокер) должен иметь реальный IP. Позволяет обойтись без VPN и пробрасывания портов.

Удобная адресация — поля данных имеют текстовые названия, понятные для человека. Не нужно запоминать цифровые адреса и битовые смещения.

Протокол MQTT работает на прикладном уровне поверх TCP/IP и использует по умолчанию 1883 порт (8883 при подключении через SSL).

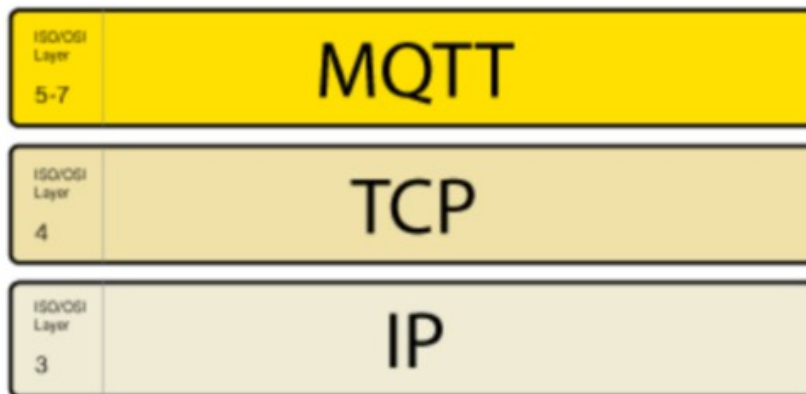


Рисунок 1 — Уровни модели OSI

Описание хода выполнения работы

Часть 1. SSH подключение

Запустим **PuTTY** для подключения к стенду по ssh, после подключения увидим приветственный баннер(рис. 2).

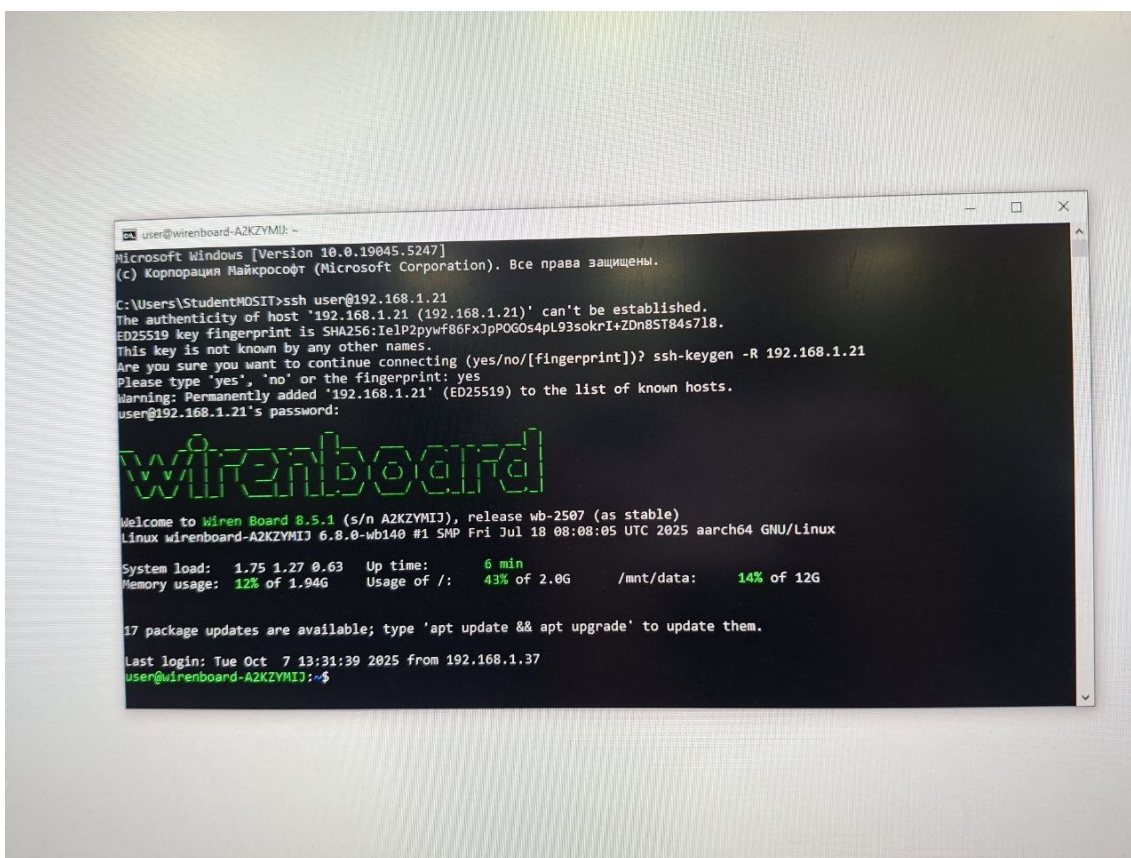


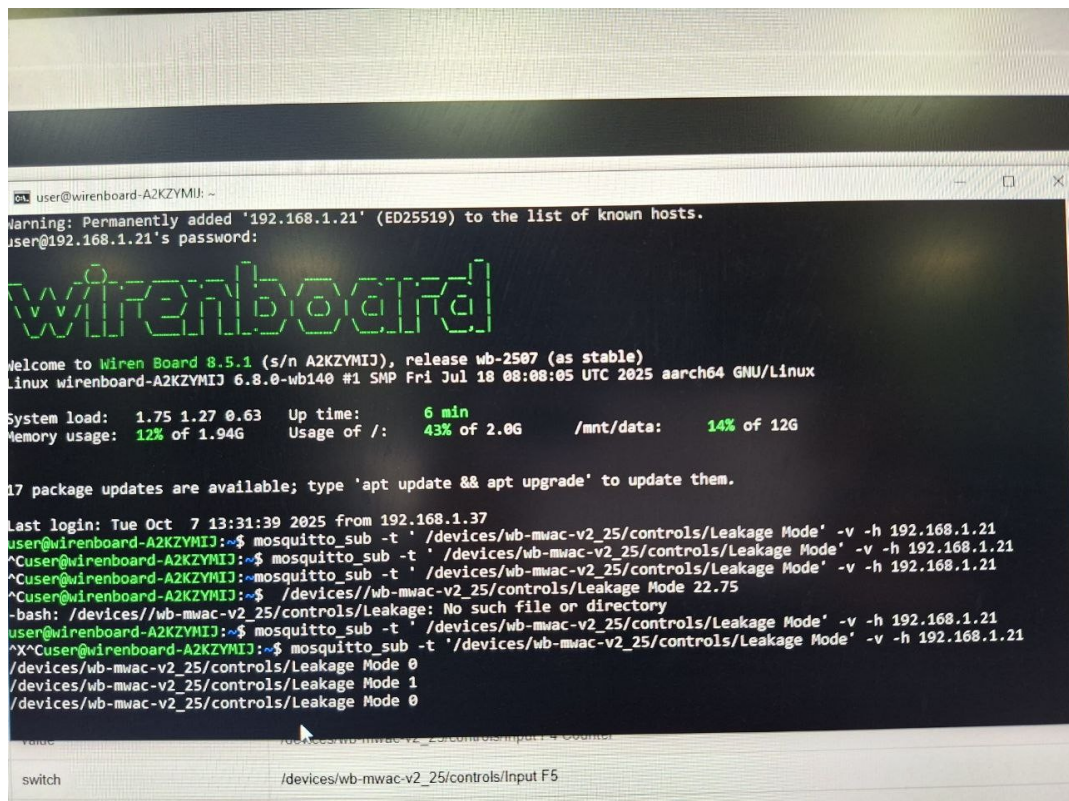
Рисунок 2 — Подключение к серверу по ssh клиенту

Часть 2. Подписка на топик

Настроим подписки на устройства стенда, которые нам понадобятся при помощи улиты `mosquitto_sub`. (рис. 3)

Согласно варианту сделаем каждое задание на своих стендах:

1. Датчик движения устройства WB-MSW v.3 (5) — стенд `wb-demo-kit v2`
2. Датчик протечки WB-MWAC v.2 — стенд `wb-demo-kit v3`



```
user@wiroreboard-A2KZYMIJ:~$  
Warning: Permanently added '192.168.1.21' (ED25519) to the list of known hosts.  
user@192.168.1.21's password:  
wiroreboard  
Welcome to Wirore Board 8.5.1 (s/n A2KZYMIJ), release wb-2507 (as stable)  
Linux wiroreboard-A2KZYMIJ 6.8.0-wb140 #1 SMP Fri Jul 18 08:08:05 UTC 2025 aarch64 GNU/Linux  
System load:  1.75 1.27 0.63   Up time:      6 min  
Memory usage: 12% of 1.94G   Usage of /:   43% of 2.0G   /mnt/data:   14% of 12G  
17 package updates are available; type 'apt update && apt upgrade' to update them.  
Last login: Tue Oct  7 13:31:39 2025 from 192.168.1.37  
user@wiroreboard-A2KZYMIJ:~$ mosquitto_sub -t '/devices/wb-mwac-v2_25/controls/Leakage Mode' -v -h 192.168.1.21  
^Cuser@wiroreboard-A2KZYMIJ:~$ mosquitto_sub -t '/devices/wb-mwac-v2_25/controls/Leakage Mode' -v -h 192.168.1.21  
^Cuser@wiroreboard-A2KZYMIJ:~$ mosquitto_sub -t '/devices/wb-mwac-v2_25/controls/Leakage Mode' -v -h 192.168.1.21  
^Cuser@wiroreboard-A2KZYMIJ:~$ /devices/wb-mwac-v2_25/controls/Leakage Mode 22.75  
-bash: /devices/wb-mwac-v2_25/controls/Leakage: No such file or directory  
user@wiroreboard-A2KZYMIJ:~$ mosquitto_sub -t '/devices/wb-mwac-v2_25/controls/Leakage Mode' -v -h 192.168.1.21  
^Cuser@wiroreboard-A2KZYMIJ:~$ mosquitto_sub -t '/devices/wb-mwac-v2_25/controls/Leakage Mode' -v -h 192.168.1.21  
/devices/wb-mwac-v2_25/controls/Leakage Mode 0  
/devices/wb-mwac-v2_25/controls/Leakage Mode 1  
/devices/wb-mwac-v2_25/controls/Leakage Mode 0  
switch /devices/wb-mwac-v2_25/controls/Input F5
```

Рисунок 3 — Подключенная подписка на стенде `wb2`

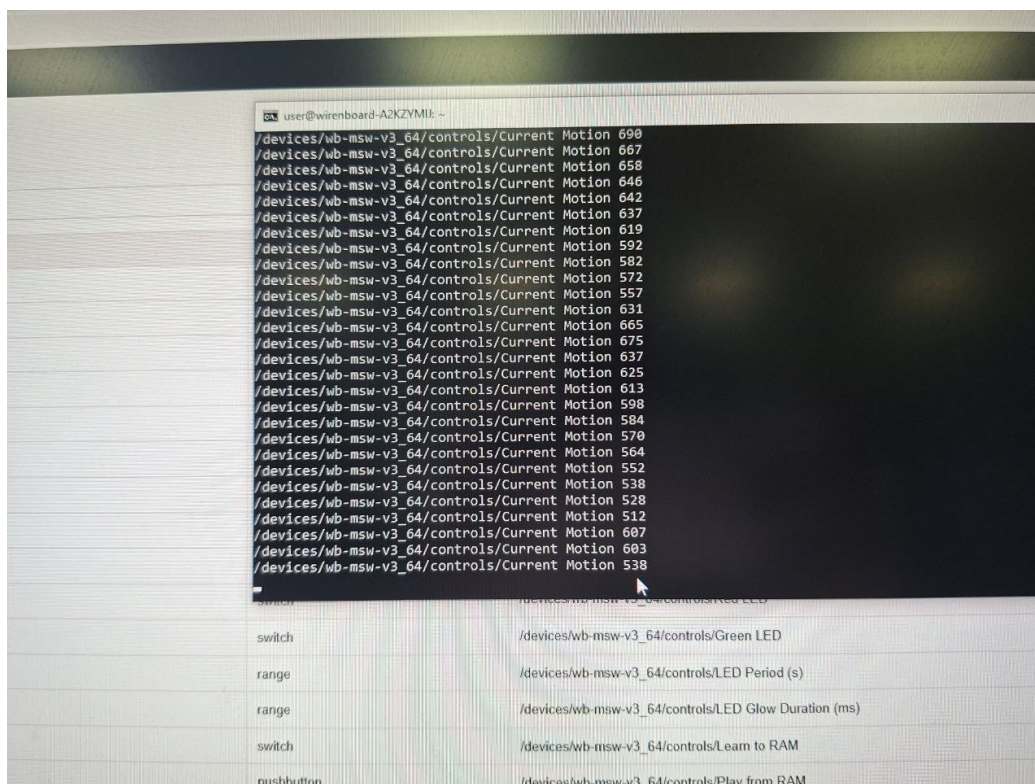


Рисунок 4 — Подключенная подписка на стенде wb3

Часть 3. Управление устройствами

Теперь включим управления устройством отправив команду с топиком:

Листинг 1 — Топик для ввода в команду.

/devices/<device-id>/controls/<control-id>/on

Пример команды:

Листинг 2 — Пример команды для управления устройством

~# mosquitto_pub -t "/devices/wb-mrm2_130/controls/Relay 1/on" -m "1" -p 1883

Согласна варианту изменим управление компонентов:

1. Включите подсветку кнопки 29 - стенд wb-demo-kit v2
2. Изменение насыщенности RGB-ленты - стенд wb-demo-kit v3

Стенд wb-demo-kit v2:


```
irenboard-A2KZYMI1:~$  
wb-msw-v3_64/controls/Current Motion 729  
wb-msw-v3_64/controls/Current Motion 713  
wb-msw-v3_64/controls/Current Motion 688  
wb-msw-v3_64/controls/Current Motion 683  
wb-msw-v3_64/controls/Current Motion 659  
wb-msw-v3_64/controls/Current Motion 641  
wb-msw-v3_64/controls/Current Motion 638  
wb-msw-v3_64/controls/Current Motion 617  
wb-msw-v3_64/controls/Current Motion 599  
wb-msw-v3_64/controls/Current Motion 598  
wb-msw-v3_64/controls/Current Motion 729  
wb-msw-v3_64/controls/Current Motion 785  
wb-msw-v3_64/controls/Current Motion 765  
wb-msw-v3_64/controls/Current Motion 756  
wb-msw-v3_64/controls/Current Motion 742  
wb-msw-v3_64/controls/Current Motion 729  
wb-msw-v3_64/controls/Current Motion 727  
wb-msw-v3_64/controls/Current Motion 712  
wb-msw-v3_64/controls/Current Motion 697  
wb-msw-v3_64/controls/Current Motion 684  
wb-msw-v3_64/controls/Current Motion 678  
wb-msw-v3_64/controls/Current Motion 661  
wb-msw-v3_64/controls/Current Motion 655  
wb-msw-v3_64/controls/Current Motion 648  
wb-msw-v3_64/controls/Current Motion 629  
irenboard-A2KZYMI1:~$ mosquitto_pub -t '/devices/wb-mr6c_46/controls/K3/on' -m '1' -p 1883  
mosquitto_o_pub: command not found  
irenboard-A2KZYMI1:~$ mosquitto_pub -t '/devices/wb-mr6c_46/controls/K3/on' -m '1' -p 1883  
irenboard-A2KZYMI1:~$
```

Рисунок 5 — Включаем кнопку



Рисунок 6 — Включенная кнопка

Стенд wb-demo-kit v3:

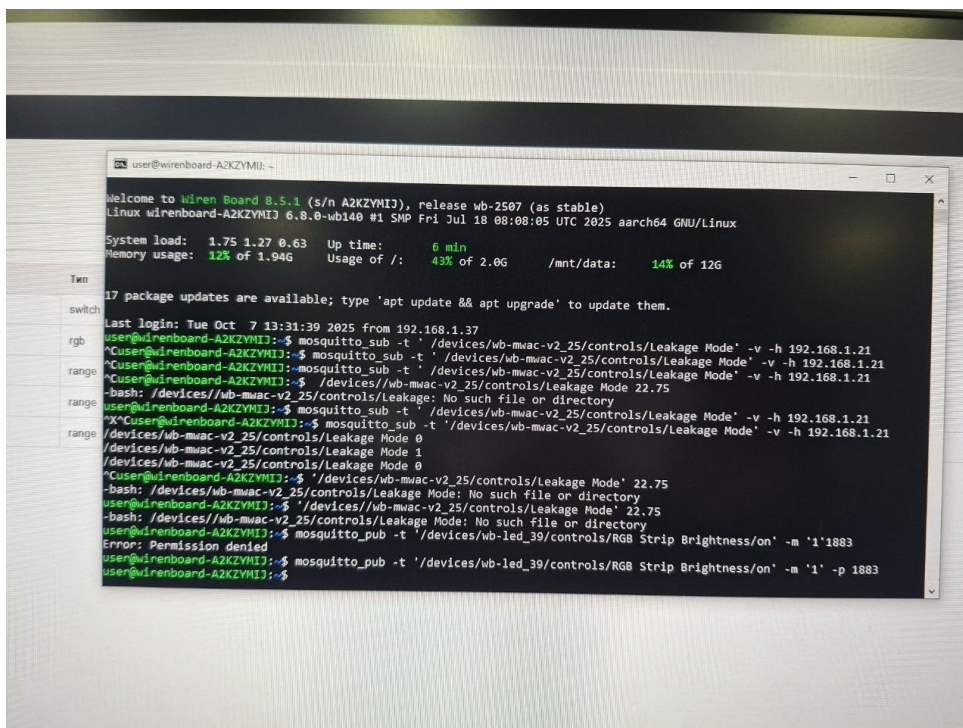
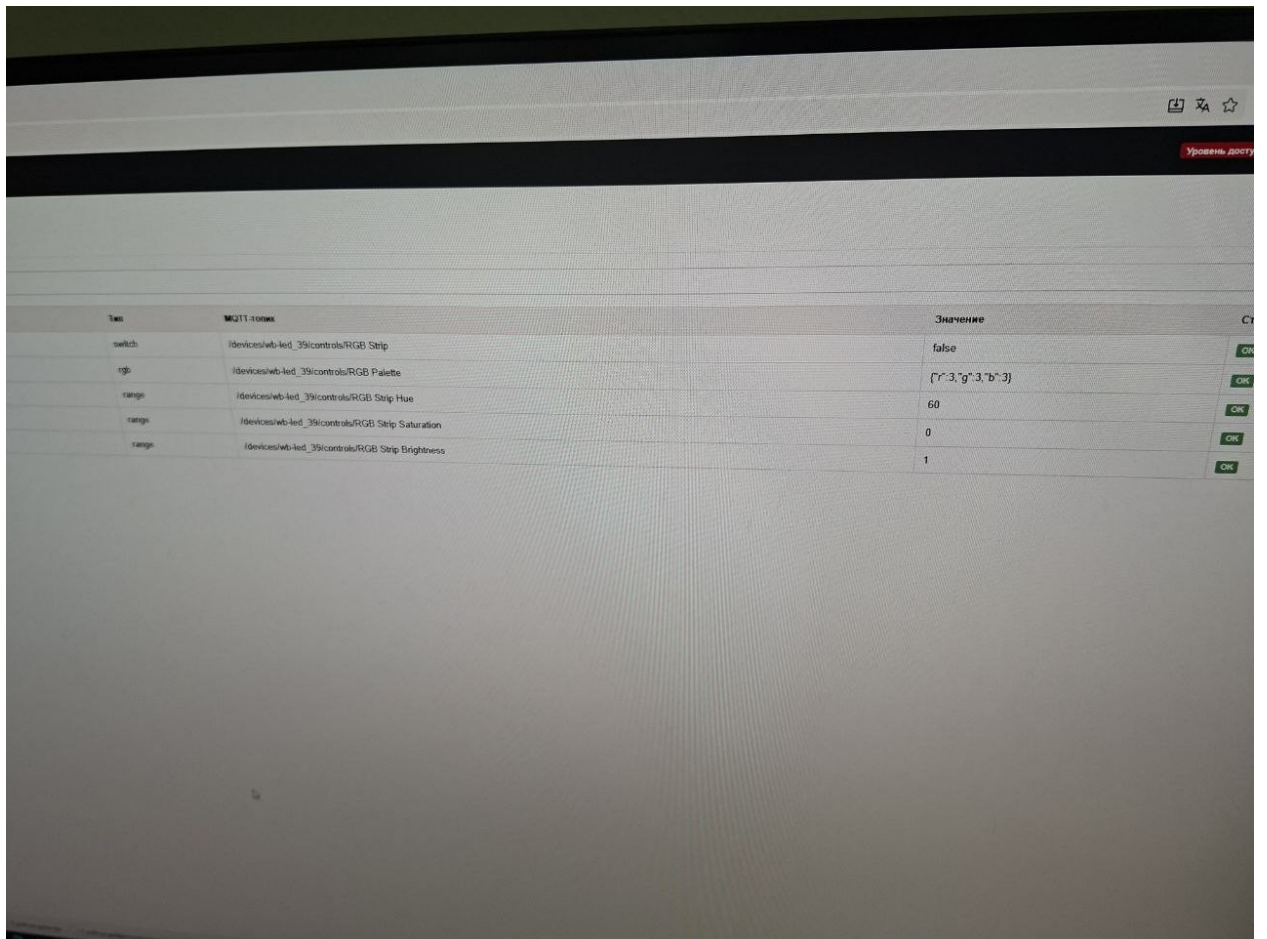


Рисунок
7 —

Включаем RGB

На рисунках 8-9 показана включенная grb подсветка в web интерфейсе:



Тип	MQTT-топик	Значение	Статус
switch	/devices/web-led_39/controls/RGB Strip	false	OK
rgb	/devices/web-led_39/controls/RGB Palette	["r":3,"g":3,"b":3]	OK
range	/devices/web-led_39/controls/RGB Strip Hue	60	OK
range	/devices/web-led_39/controls/RGB Strip Saturation	0	OK
range	/devices/web-led_39/controls/RGB Strip Brightness	1	OK

Рисунок 8 — RGB подцветка(часть 1)

Тип	MQTT-топик	Значение	Статус
switch	/devices/wb-led_39/controls/RGB Strip	false	OK
rgb	/devices/wb-led_39/controls/RGB Palette	[r:255,g:255,b:255]	OK
range	/devices/wb-led_39/controls/RGB Strip Hue	60	OK
range	/devices/wb-led_39/controls/RGB Strip Saturation	0	OK
range	/devices/wb-led_39/controls/RGB Strip Brightness	100	OK

Рисунок 9 — RGB подцветка(часть 2)

Практическая работа №7 - Форматы представления данных

Краткое введение:

Системы Интернета вещей часто характеризуется большим количеством разнородных компонентов, и для корректного взаимодействия этих компонентов(т.е. для обмена данным без потерь и повреждения) необходимо поддерживать синтаксическую совместимость.

Некоторые форматы представления данных особенно хорошо подходят для обмена данными между компонентами системы Интернета вещей и между различными системами. Примером таких форматов представления данных может служить XML (расширяемый язык разметки) и JSON.

Описание работы:

Стенд wb-2

```
SUB_TOPICS = {  
    '/devices/wb-map12e_23/controls/Ch 1 P L2': 'power',  
    '/devices/wb-msw-v3_21/controls/Current Motion': 'motion',  
    '/devices/wb-ms_11/controls/Temperature': 'temperature',  
    '/devices/wb-msw-v3_21/controls/Sound Level': 'sound'  
}
```

Рисунок 10 – Команды на подписки на топики

The screenshot shows a code editor with a file named `paho_test_commented.py`. The code is a Python script that interacts with an MQTT broker. It includes a menu for users to choose actions like starting/stopping data collection, showing data, exporting to XML, and statistics. The execution output at the bottom shows the program running, connecting to the MQTT broker, and displaying various sensor data points such as temperature, sound level, and motion status for different devices.

```
224     """Оформление выходных данных"""
225     print("\n" + "-" * 40)
226     print("      MQTT ДАТЧИКИ - СБОРЩИК И ПАРСЕР")
227     print("-" * 40)
228     print("УПРАВЛЕНИЕ СБОРЩИКОМ:")
229     print(" 1 - Запустить сборщик данных")
230     print(" 2 - Остановить сборщик данных")
231     print(" 3 - Статус сборщика")
232     print("\n РАБОТА С ДАННЫМИ:")
233     print(" 4 - Показать все данные")
234     print(" 5 - Показать последние записи")
235     print(" 6 - Экспорт в XML")
236     print(" 7 - Статистика")
237     print("\n 0 - Выход")
238     print("-" * 40)
239
240 def main():
241     """Основная функция программы"""
242     print("MQTT Система мониторинга датчиков")
243     print("Сборщик + Парсер в одной программе")
244
245     while True:
246         show_menu()
247         choice = input("Выберите действие (0-7): ").strip()
248
249         if choice == "1":
250             # Запустить сборщик данных
251             start_data_collection()
252
253         elif choice == "2":
254             # Остановить сборщик данных
255             stop_data_collection()
256
257         elif choice == "3":
258             # Статус сборщика
259             get_status()
260
261         elif choice == "4":
262             # Показать все данные
263             show_all_data()
264
265         elif choice == "5":
266             # Показать последние записи
267             show_last_records()
268
269         elif choice == "6":
270             # Экспорт в XML
271             export_to_xml()
272
273         elif choice == "7":
274             # Статистика
275             show_statistics()
276
277         elif choice == "0":
278             break
279
280 if __name__ == "__main__":
281     main()
```

Run paho_test_commented x

```
Выберите действие (0-7): 1
Запуск сборщика MQTT данных...
Сборщик запущен. Данные сохраняются в data.json

Нажмите Enter для продолжения...D:\paho_test_commented.py:76: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
client = mqtt.Client()
Connected with result code 0
Subscribed to: /devices/wb-map12e_23/controls/Ch 1 P L2
Subscribed to: /devices/wb-msw-v3_21/controls/Current Motion
Subscribed to: /devices/wb-ms_11/controls/Temperature
Subscribed to: /devices/wb-msw-v3_21/controls/Sound Level
/devices/wb-map12e_23/controls/Ch 1 P L2: 0.15
/devices/wb-msw-v3_21/controls/Current Motion: 272
/devices/wb-ms_11/controls/Temperature: 26.46
/devices/wb-msw-v3_21/controls/Sound Level: 71.63
/devices/wb-msw-v3_21/controls/Sound Level: 62.07
/devices/wb-msw-v3_21/controls/Current Motion: 152
/devices/wb-map12e_23/controls/Ch 1 P L2: 0.13

===== MQTT ДАТЧИКИ - СБОРЩИК И ПАРСЕР =====
УПРАВЛЕНИЕ СБОРЩИКОМ:
1 - Запустить сборщик данных
2 - Остановить сборщик данных
```

Рисунок 11 – Структура данных вывода

Листинг 3 – Код на языке python

```
import paho.mqtt.client as mqtt

import json

import xml.etree.ElementTree as ET

from datetime import datetime

import threading

import os


# Параметры подключения к MQTT-брокеру

HOST = "192.168.1.17"

PORT = 1883
```

```
KEEPALIVE = 17
```

```
# Словарь с топиками и собираемыми из них параметрами
```

```
SUB_TOPICS = {
```

```
    '/devices/wb-map12e_23/controls/Ch 1 P L2': 'power',
```

```
    '/devices/wb-msw-v3_21/controls/Current Motion': 'motion',
```

```
    '/devices/wb-ms_11/controls/Temperature': 'temperature',
```

```
    '/devices/wb-msw-v3_21/controls/Sound Level': 'sound'
```

```
}
```

```
# Глобальные переменные
```

```
JSON_LIST = []
```

```
JSON_DICT = {}
```

```
for value in SUB_TOPICS.values():
```

```
    JSON_DICT[value] = 0
```

```
# Флаги для управления потоками
```

```
collector_running = False
```

```
client = None
```

```
#
```

```
=====
```

```
=====
```

```
# ЧАСТЬ 1: СБОРЩИК ДАННЫХ (ваш оригинальный код + улучшения)
```

```
#
```

```

=====
=====

def on_connect(client, userdata, flags, rc):
    """Функция, вызываемая при подключении к брокеру"""
    print("✅ Connected with result code " + str(rc))
    for topic in SUB_TOPICS.keys():
        client.subscribe(topic)
        print(f"📡 Subscribed to: {topic}")

def on_message(client, userdata, msg):
    """Функция, вызываемая при получении сообщения от брокера"""
    payload = msg.payload.decode()
    topic = msg.topic

    param_name = SUB_TOPICS[topic]
    JSON_DICT[param_name] = payload
    JSON_DICT['time'] = str(datetime.now())

    JSON_LIST.append(JSON_DICT.copy())

    print(f"✉️ {topic}: {payload}")

# Автосохранение при каждом сообщении (как в вашем коде)

```

```
with open('data.json', 'w') as file:
```

```
    json_string = json.dumps(JSON_LIST)
```

```
    file.write(json_string)
```

```
def start_collector():
```

```
    """Запуск сборщика данных в отдельном потоке"""
```

```
    global collector_running, client
```

```
    if collector_running:
```

```
        print("⚠ Сборщик уже запущен!")
```

```
        return
```

```
    print("🚀 Запуск сборщика MQTT данных...")
```

```
    collector_running = True
```

```
def collector_thread():
```

```
    global client
```

```
    client = mqtt.Client()
```

```
    client.on_connect = on_connect
```

```
    client.on_message = on_message
```

```
    try:
```

```
        client.connect(HOST, PORT, KEEPALIVE)
```

```
        client.loop_forever()
```

```
except Exception as e:
```

```
    print(f"❌ Ошибка подключения: {e}")
```

```
    collector_running = False
```

```
thread = threading.Thread(target=collector_thread, daemon=True)
```

```
thread.start()
```

```
print("✅ Сборщик запущен. Данные сохраняются в data.json")
```

```
def stop_collector():
```

```
    """Остановка сборщика данных"""
```

```
    global collector_running, client
```

```
    if client:
```

```
        client.disconnect()
```

```
        collector_running = False
```

```
        print("🛑 Сборщик остановлен")
```

```
    else:
```

```
        print("⚠️ Сборщик не запущен")
```

```
#
```

```
=====
```

```
# ЧАСТЬ 2: ПАРСЕР ДАННЫХ
```

```
#
```

```
=====
=====

def parse_all_data():
    """Парсинг всех данных из JSON файла"""
    print("\n🔍 ПАРСИНГ ВСЕХ ДАННЫХ")
    print("=" * 60)

    try:
        with open('data.json', 'r', encoding='utf-8') as file:
            data_list = json.load(file)

        if not data_list:
            print("❌ Файл пуст или не содержит данных")
            return

        print(f"📊 Найдено записей: {len(data_list)}")
        print("=" * 60)

        for i, data in enumerate(data_list, 1):
            print(f"\n📄 Запись #{i}")
            print("-" * 40)
            display_sensor_data(data)

    except FileNotFoundError:
```



```
print("❌ Файл data.json не найден")
```

```
except Exception as e:
```

```
print(f"❌ Ошибка: {e}")
```

```
def parse_last_record():
```

```
    """Парсинг последней записи"""
```

```
    print("\n🎯 ПОСЛЕДНЯЯ ЗАПИСЬ")
```

```
    print("=" * 50)
```

```
    try:
```

```
        with open('data.json', 'r', encoding='utf-8') as file:
```

```
            data_list = json.load(file)
```

```
        if data_list:
```

```
            display_sensor_data(data_list[-1])
```

```
        else:
```

```
            print("❌ Нет данных")
```

```
    except FileNotFoundError:
```

```
        print("❌ Файл data.json не найден")
```

```
    except Exception as e:
```

```
        print(f"❌ Ошибка: {e}")
```

```
def display_sensor_data(data):
```

```
"""Отображение данных датчиков"""
```

```
print(f"⚡ Потребляемая мощность: {data.get('power', 'N/A')}")
```

```
print(f"🎯 Датчик движения: {data.get('motion', 'N/A')}")
```

```
print(f"🌡 Датчик температуры: {data.get('temperature', 'N/A')}")
```

```
print(f"🔊 Уровень звука: {data.get('sound', 'N/A')}")
```

```
print(f"🕒 Время записи: {data.get('time', 'N/A')}")
```

```
print("-" * 40)
```

```
def export_to_xml():
```

```
    """Экспорт данных в XML"""
```

```
    try:
```

```
        with open('data.json', 'r', encoding='utf-8') as file:
```

```
            data_list = json.load(file)
```

```
    if not data_list:
```

```
        print("❌ Нет данных для экспорта")
```

```
        return
```

```
    root = ET.Element('sensor_data_collection')
```

```
    for i, data in enumerate(data_list, 1):
```

```
        record_element = ET.SubElement(root, 'record', id=str(i))
```

```
        for key, value in data.items():
```

```
            field_element = ET.SubElement(record_element, key)
```

```
field_element.text = str(value)
```

```
xml_filename =  
f'sensor_export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.xml'
```

```
tree = ET.ElementTree(root)
```

```
tree.write(xml_filename, encoding='utf-8', xml_declaration=True)
```

```
print(f"✅ Данные экспортированы в {xml_filename}")
```

```
print(f"📊 Записей: {len(data_list)}")
```

```
except Exception as e:
```

```
print(f"❌ Ошибка экспорта: {e}")
```

```
def show_statistics():
```

```
    """Показать статистику по данным"""
```

```
    try:
```

```
        with open('data.json', 'r', encoding='utf-8') as file:
```

```
            data_list = json.load(file)
```

```
    if not data_list:
```

```
        print(f"❌ Нет данных для статистики")
```

```
        return
```

```
print(f"\n📈 СТАТИСТИКА ДАННЫХ")
```

```
print("=" * 40)
```

```
print(f" Всего записей: {len(data_list)}")
```

```
if len(data_list) >= 2:
```

```
    first_time = data_list[0].get('time', "")
```

```
    last_time = data_list[-1].get('time', "")
```

```
    print(f" Первая запись: {first_time}")
```

```
    print(f" Последняя запись: {last_time}")
```

```
except Exception as e:
```

```
    print(f" Ошибка: {e}")
```

```
#
```

```
=====
=====
```

```
# ОСНОВНОЕ МЕНЮ И УПРАВЛЕНИЕ
```

```
#
```

```
=====
=====
```

```
def show_menu():
```

```
    """Отображение главного меню"""
```

```
    print("\n" + "=" * 60)
```

```
    print("       MQTT ДАТЧИКИ - СБОРЩИК И ПАРСЕР")
```

```
    print("=" * 60)
```

```
print("📡 УПРАВЛЕНИЕ СБОРЩИКОМ:")
print(" 1 - Запустить сборщик данных")
print(" 2 - Остановить сборщик данных")
print(" 3 - Статус сборщика")
print("\n🔍 РАБОТА С ДАННЫМИ:")
print(" 4 - Показать все данные")
print(" 5 - Показать последнюю запись")
print(" 6 - Экспорт в XML")
print(" 7 - Статистика")
print("\n 0 - Выход")
print("-" * 60)
```

```
def main():
```

```
    """Основная функция программы"""
```

```
    print("🚀 MQTT Система мониторинга датчиков")
```

```
    print("💡 Сборщик + Парсер в одной программе")
```

```
    while True:
```

```
        show_menu()
```

```
        choice = input("Выберите действие (0-7): ").strip()
```

```
        if choice == "1":
```

```
            start_collector()
```

```
        elif choice == "2":
```

```
stop_collector()

elif choice == "3":

    status = "✅ Запущен" if collector_running else "❌ Остановлен"

    print(f"📊 Статус сборщика: {status}")

    if os.path.exists('data.json'):

        try:

            with open('data.json', 'r') as f:

                data = json.load(f)

                print(f"📁 Записей в файле: {len(data)}")

        except:

            print(f"📁 Файл данных: есть (ошибка чтения)")

    else:

        print(f"📁 Файл данных: отсутствует")

elif choice == "4":

    parse_all_data()

elif choice == "5":

    parse_last_record()

elif choice == "6":

    export_to_xml()

elif choice == "7":

    show_statistics()

elif choice == "0":

    print(f"👋 Выход из программы...")

    stop_collector()

    break
```

```
else:
```

```
    print("❌ Неверный выбор, попробуйте снова")
```

```
    input("\n↵ Нажмите Enter для продолжения...")
```

```
if __name__ == "__main__":
```

```
    main()
```

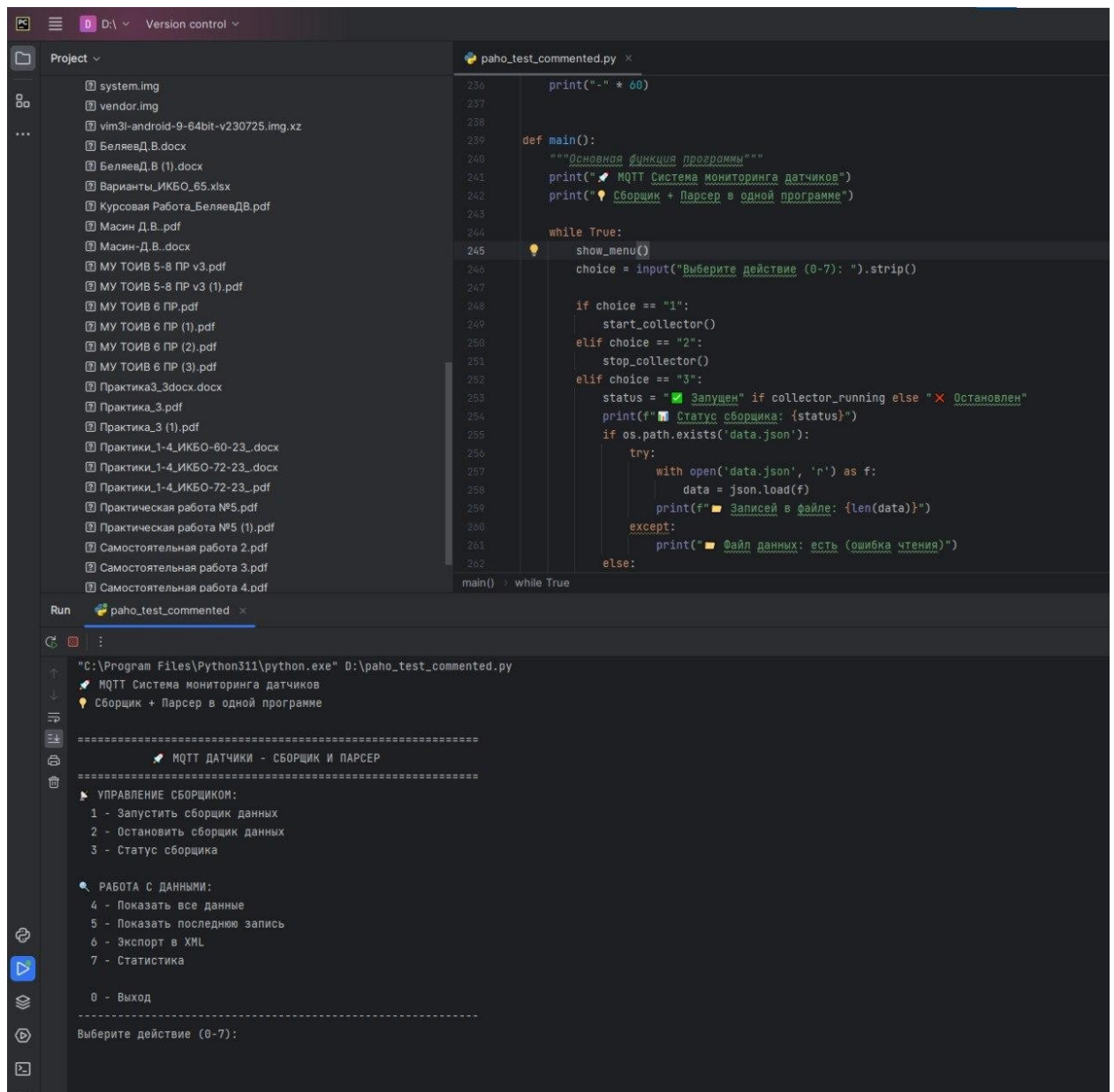


Рисунок 12 – Работа программы(часть 1)

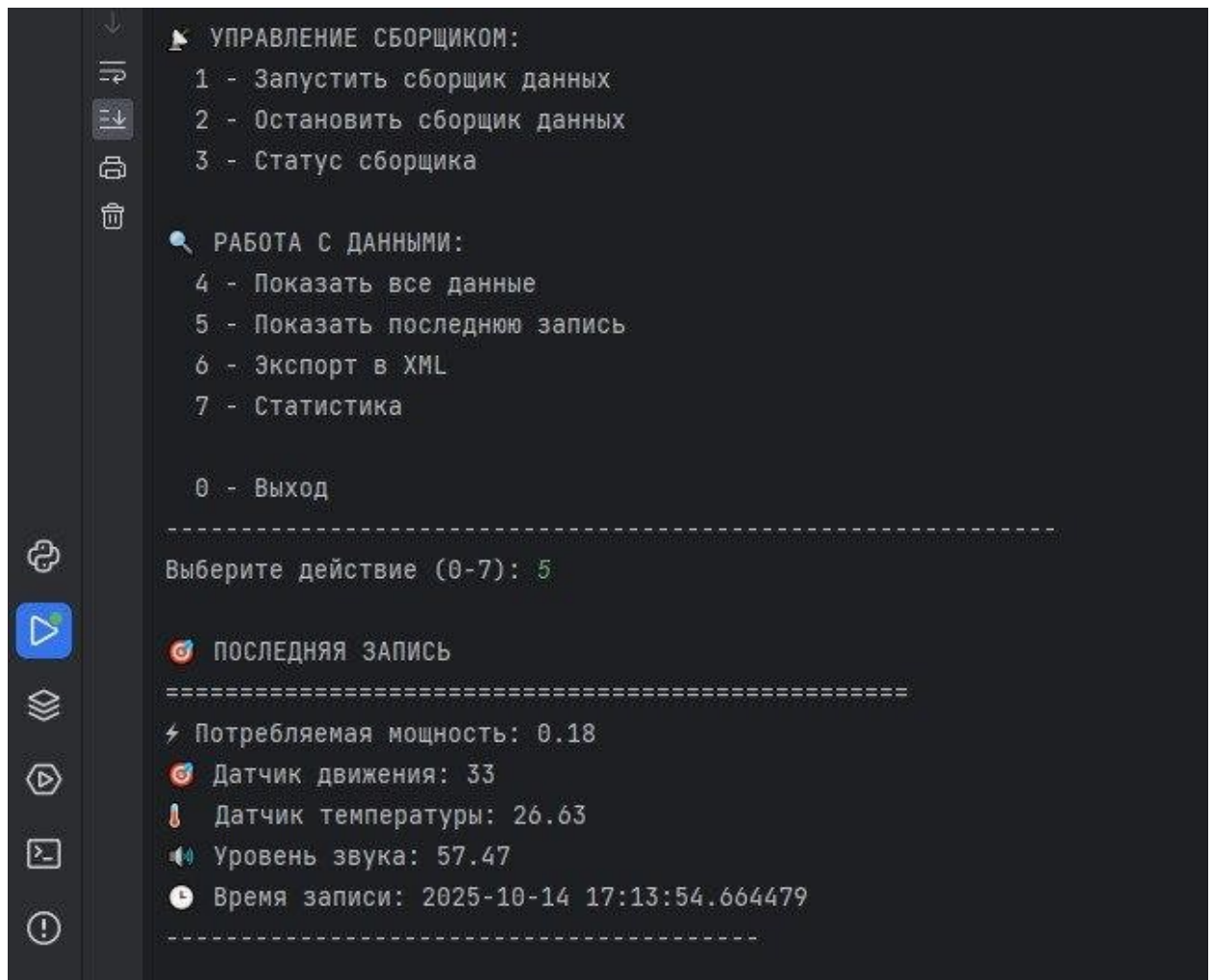


Рисунок 13 - Работа программы(часть 2)

```

import paho.mqtt.client as mqtt
import json
import xml.etree.ElementTree as ET
from datetime import datetime
import threading
import os

```

Рисунок 14 – Библиотеки используемые в программе

Стенд wb-3:

```

SUB_TOPICS = {
    '/devices/wb-msw-v3_64/controls/Current Motion': 'motion',
    'devices/wb-msw-v3_64/controls/Temperature': 'temperature',
    '/devices/wb-map12e_35/controls/Ch 3 P L1': 'voltage'
}

```

Рисунок 15 – Команды подписки на топики

```

✅ Сборщик запущен. Данные сохраняются в data.json

⏏ Нажмите Enter для продолжения...✅ Connected with result code 0
📁 Subscribed to: /devices/wb-msw-v3_64/controls/Current Motion
📁 Subscribed to: devices/wb-msw-v3_64/controls/Temperature
📁 Subscribed to: /devices/wb-map12e_35/controls/Ch 3 P L1
📁 /devices/wb-msw-v3_64/controls/Current Motion: 59
📁 /devices/wb-map12e_35/controls/Ch 3 P L1: 0
📁 /devices/wb-msw-v3_64/controls/Current Motion: 53
📁 /devices/wb-msw-v3_64/controls/Current Motion: 53
📁 /devices/wb-msw-v3_64/controls/Current Motion: 49
📁 Data saved to XML: sensor_data_20251014_173018.xml
📁 /devices/wb-msw-v3_64/controls/Current Motion: 49
📁 /devices/wb-msw-v3_64/controls/Current Motion: 43
📁 /devices/wb-msw-v3_64/controls/Current Motion: 43
📁 /devices/wb-msw-v3_64/controls/Current Motion: 38
📁 /devices/wb-msw-v3_64/controls/Current Motion: 38
📁 Data saved to XML: sensor_data_20251014_173019.xml
📁 /devices/wb-msw-v3_64/controls/Current Motion: 47
📁 /devices/wb-msw-v3_64/controls/Current Motion: 47
📁 /devices/wb-msw-v3_64/controls/Current Motion: 39
📁 /devices/wb-msw-v3_64/controls/Current Motion: 39
📁 /devices/wb-msw-v3_64/controls/Current Motion: 36
📁 Data saved to XML: sensor_data_20251014_173020.xml
📁 /devices/wb-msw-v3_64/controls/Current Motion: 36
📁 /devices/wb-msw-v3_64/controls/Current Motion: 34
📁 /devices/wb-msw-v3_64/controls/Current Motion: 34
2 📁 /devices/wb-msw-v3_64/controls/Current Motion: 30
📁 /devices/wb-msw-v3_64/controls/Current Motion: 30
📁 Data saved to XML: sensor_data_20251014_173021.xml

```

Рисунок 16 - Структура данных вывода

Листинг 4 – Код на языке python

```
import paho.mqtt.client as mqtt

import json

import xml.etree.ElementTree as ET

from datetime import datetime

import socket

import threading

import time

import os


# Параметры подключения

HOST = "192.168.1.27" # IP стенда

PORT = 1883

KEEPALIVE = 17


# Топики для WB-demo-kit v.3

SUB_TOPICS = {

    '/devices/wb-msw-v3_64/controls/Current Motion': 'motion',

    '/devices/wb-msw-v3_64/controls/Temperature': 'temperature',

    '/devices/wb-map12e_35/controls/Ch 3 P L1': 'voltage'

}


JSON_LIST = []

JSON_DICT = {}

for value in SUB_TOPICS.values():
```

```
JSON_DICT[value] = 0

# Флаги для управления
collector_running = False
client = None


#
=====
=====

# СБОРЩИК ДАННЫХ
#
=====
=====


def get_suitcase_number():
    """Получение номера чемодана (последние две цифры IP)"""
    try:
        hostname = socket.gethostname()
        local_ip = socket.gethostbyname(hostname)
        return local_ip.split('.')[-1]
    except:
        return "27" # Запасное значение


def on_connect(client, userdata, flags, rc):
```

```
print("✅ Connected with result code " + str(rc))
```

```
for topic in SUB_TOPICS.keys():
```

```
    client.subscribe(topic)
```

```
    print(f"📡 Subscribed to: {topic}")
```

```
def on_message(client, userdata, msg):
```

```
    payload = msg.payload.decode()
```

```
    topic = msg.topic
```

```
    param_name = SUB_TOPICS[topic]
```

```
    JSON_DICT[param_name] = payload
```

```
    JSON_DICT['time'] = str(datetime.now())
```

```
    JSON_DICT['suitcase_number'] = get_suitcase_number()
```

```
    JSON_LIST.append(JSON_DICT.copy())
```

```
    print(f"✉️ {topic}: {payload}")
```

```
# Сохранение в JSON
```

```
with open('data.json', 'w') as file:
```

```
    json.dump(JSON_LIST, file)
```

```
# Сохранение в XML каждые 5 сообщений
```

```
if len(JSON_LIST) % 5 == 0:
```

```
save_to_xml()
```

```
def save_to_xml():
```

```
    """Сохранение последней записи в XML"""
```

```
    if JSON_LIST:
```

```
        data = JSON_LIST[-1]
```

```
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
```

```
        filename = f'sensor_data_{timestamp}.xml'
```

```
        root = ET.Element('sensor_data')
```

```
        for key, value in data.items():
```

```
            element = ET.SubElement(root, key)
```

```
            element.text = str(value)
```

```
        tree = ET.ElementTree(root)
```

```
        tree.write(filename, encoding='utf-8', xml_declaration=True)
```

```
        print(f" Data saved to XML: {filename}")
```

```
def start_collector():
```

```
    """Запуск сборщика данных"""
```

```
    global collector_running, client
```

```
    if collector_running:
```

```
print("⚠ Сборщик уже запущен!")
```

```
return
```

```
print("🚀 Запуск сборщика MQTT данных...")
```

```
collector_running = True
```

```
def collector_thread():
```

```
    global client
```

```
    client = mqtt.Client()
```

```
    client.on_connect = on_connect
```

```
    client.on_message = on_message
```

```
    try:
```

```
        client.connect(HOST, PORT, KEEPALIVE)
```

```
        client.loop_forever()
```

```
    except Exception as e:
```

```
        print(f"❌ Ошибка подключения: {e}")
```

```
        collector_running = False
```

```
thread = threading.Thread(target=collector_thread, daemon=True)
```

```
thread.start()
```

```
print("✅ Сборщик запущен. Данные сохраняются в data.json")
```

```
def stop_collector():
```

```

"""Остановка сборщика данных"""

global collector_running, client

if client and collector_running:

    client.disconnect()

    collector_running = False

    print("🛑 Сборщик остановлен")
else:

    print("⚠️ Сборщик не запущен")


#
=====
=====

# ПАРСЕР

#
=====
=====


def show_sensor_data():

    """Показать данные датчиков"""

    print("\n🔍 ДАННЫЕ ДАТЧИКОВ")

    print("=" * 50)


try:

    if not os.path.exists('data.json'):

```



```
print("❌ Файл data.json не найден")
```

```
return
```

```
with open('data.json', 'r', encoding='utf-8') as file:
```

```
    data_list = json.load(file)
```

```
if not data_list:
```

```
    print("❌ Нет данных в файле")
```

```
    return
```

```
print(f"📊 Всего записей: {len(data_list)}\n")
```

```
# Показать последние 5 записей
```

```
print("📋 ПОСЛЕДНИЕ ЗАПИСИ:")
```

```
print("=" * 50)
```

```
start_index = max(0, len(data_list) - 5)
```

```
for i, data in enumerate(data_list[start_index:], start_index + 1):
```

```
    print(f"\n📄 Запись #{i}:")
```

```
    print(f"🎯 Движение: {data.get('motion', 'N/A')}")
```

```
    print(f"🌡️ Температура: {data.get('temperature', 'N/A')}")
```

```
    print(f"⚡ Напряжение: {data.get('voltage', 'N/A')}")
```

```
    print(f"🕒 Время: {data.get('time', 'N/A')}")
```

```
    print(f"🧳 Чемодан: {data.get('suitcase_number', 'N/A')}")
```

```
    print("-" * 30)
```

```
except Exception as e:
```

```
    print(f"❌ Ошибка чтения данных: {e}")
```

```
def show_last_record():
```

```
    """Показать только последнюю запись"""
```

```
    print("\n🎯 ПОСЛЕДНЯЯ ЗАПИСЬ")
```

```
    print("=" * 40)
```

```
    try:
```

```
        if not os.path.exists('data.json'):
```

```
            print("❌ Файл data.json не найден")
```

```
            return
```

```
        with open('data.json', 'r', encoding='utf-8') as file:
```

```
            data_list = json.load(file)
```

```
        if data_list:
```

```
            data = data_list[-1]
```

```
            print(f"🎯 Движение: {data.get('motion', 'N/A')}")
```

```
            print(f"🌡️ Температура: {data.get('temperature', 'N/A')}")
```

```
            print(f"⚡ Напряжение: {data.get('voltage', 'N/A')}")
```

```
            print(f"🕒 Время: {data.get('time', 'N/A')}")
```

```
            print(f"🧳 Чемодан: {data.get('suitcase_number', 'N/A')}")
```

else:

```
print("❌ Нет данных")
```

except Exception as e:

```
print(f"❌ Ошибка: {e}")
```

def show_statistics():

```
"""Показать простую статистику"""
```

```
print("\n📊 СТАТИСТИКА")
```

```
print("=" * 40)
```

try:

```
if not os.path.exists('data.json'):
```

```
    print("❌ Файл data.json не найден")
```

```
    return
```

```
with open('data.json', 'r', encoding='utf-8') as file:
```

```
    data_list = json.load(file)
```

```
if not data_list:
```

```
    print("❌ Нет данных")
```

```
    return
```

```
print(f"📈 Всего записей: {len(data_list)}")
```

```
# Статистика по движению
```

```
motion_count = sum(1 for data in data_list if data.get('motion') == '1')
```

```
print(f"🎯 Движение обнаружено: {motion_count} раз")
```

```
# Последние значения
```

```
if data_list:
```

```
    last = data_list[-1]
```

```
    print(f"🌡️ Текущая температура: {last.get('temperature', 'N/A')}")
```

```
    print(f"⚡ Текущее напряжение: {last.get('voltage', 'N/A')}")
```

```
except Exception as e:
```

```
    print(f"❌ Ошибка: {e}")
```

```
def show_xml_files():
```

```
    """Показать созданные XML файлы"""
```

```
    print("\n📁 XML ФАЙЛЫ")
```

```
    print("=" * 40)
```

```
    xml_files = [f for f in os.listdir('.') if f.startswith('sensor_data_') and  
f.endswith('.xml')]
```

```
    if not xml_files:
```

```
        print(f"❌ XML файлы не найдены")
```

```
return
```

```
print(f"📄 Найдено XML файлов: {len(xml_files)}")
```

```
for xml_file in sorted(xml_files):
```

```
    print(f"📋 {xml_file}")
```

```
#
```

```
=====
=====
```

```
# ГЛАВНОЕ МЕНЮ
```

```
#
```

```
=====
=====
```

```
def show_menu():
```

```
    """Показать главное меню"""
```

```
    print("\n" + "=" * 60)
```

```
    print("🚀 WB-ДЕМО-KIT v.3 - СБОРЩИК И ПАРСЕР")
```

```
    print("=" * 60)
```

```
    print("📡 УПРАВЛЕНИЕ СБОРЩИКОМ:")
```

```
    print(" 1 - Запустить сборщик данных")
```

```
    print(" 2 - Остановить сборщик данных")
```

```
    print(" 3 - Статус сборщика")
```

```
    print("\n🔍 ПРОСМОТР ДАННЫХ:")
```

```
    print(" 4 - Показать все данные")
```

```
print(" 5 - Показать последнюю запись")  
  
print(" 6 - Статистика")  
  
print(" 7 - Список XML файлов")  
  
print("\n 0 - Выход")  
  
print("-" * 60)
```

```
def main():
```

```
    """Основная функция программы"""
```

```
    print("🚀 WB-Demo-Kit v.3 - Система мониторинга")
```

```
    print("💡 Сборщик данных + Парсер в одной программе")
```

```
    while True:
```

```
        show_menu()
```

```
        choice = input("Выберите действие (0-7): ").strip()
```

```
        if choice == "1":
```

```
            start_collector()
```

```
        elif choice == "2":
```

```
            stop_collector()
```

```
        elif choice == "3":
```

```
            status = "✅ Запущен" if collector_running else "❌ Остановлен"
```

```
            print(f"📊 Статус сборщика: {status}")
```

```
            if os.path.exists('data.json'):
```

```
                try:
```

```
        with open('data.json', 'r') as f:

            data = json.load(f)

            print(f"📁 Записей в файле: {len(data)}")

        except:

            print("📁 Файл данных: есть (ошибка чтения)")

        else:

            print("📁 Файл данных: отсутствует")

    elif choice == "4":

        show_sensor_data()

    elif choice == "5":

        show_last_record()

    elif choice == "6":

        show_statistics()

    elif choice == "7":

        show_xml_files()

    elif choice == "0":

        print("👋 Выход из программы...")

        stop_collector()

        break

    else:

        print("❌ Неверный выбор, попробуйте снова")

input("\n⬅️ Нажмите Enter для продолжения...")
```

```
if __name__ == "__main__":  
    main()
```

```
=====
WB-DEMO-KIT v.3 - СБОРЩИК И ПАРСЕР
=====
▶ УПРАВЛЕНИЕ СБОРЩИКОМ:
  1 - Запустить сборщик данных
  2 - Остановить сборщик данных
  3 - Статус сборщика

🔍 ПРОСМОТР ДАННЫХ:
  4 - Показать все данные
  5 - Показать последнюю запись
  6 - Статистика
  7 - Список XML файлов

  0 - Выход
-----
Выберите действие (0-7): 5

🔍 ПОСЛЕДНЯЯ ЗАПИСЬ
=====
🔍 Движение: 22
🔥 Температура: 0
⚡ Напряжение: 0
🕒 Время: 2025-10-14 17:30:22.289765
📦 Чемодан: 1
```

Рисунок 17 - Работа программы


```
from datetime import datetime
import socket
import threading
import os
```

Рисунок 18 – Библиотеки используемые в программе

Практическая работа №8 - Визуализация данных в Интернете вещей

Краткое введение:

Визуализация данных

После сбора показаний с датчиков и сохранения данных в системах интернета вещей зачастую предусмотрена возможность визуализации для пользователей первичных данных или результата их агрегации и анализа.

Визуализация данных — это представление данных в графической форме, обеспечивающей наиболее эффективную работу человека по их изучению, т.е. оценивания информации или данных.

Если проводить классификацию по объектам, то стоит выделить визуализацию:

- числовых данных (детерминированные зависимости – графики, диаграммы, временные ряды; статистические распределения – гистограммы, матрицы диаграмм рассеяния)
- иерархий (диаграммы узлы-связи, дендрограммы)
- сетей (графы, дуговые диаграммы)
- геовизуализацию (карты, картограммы)

В отличие от обычного графического интерфейса, средства визуализации данных обеспечивают способность одновременного отображения большого числа разнотипных данных, возможность их сравнения, выделения выпадающих значений.

Качественная визуализация данных имеет критическое значение для анализа данных и принятия решений на их основе. Продуманное графическое представление не только содержит информацию, но и повышает эффективность ее восприятия за счет наглядности.

Описание работы:

В данной работе мы соберем данные в формате csv за 10 минутный период, а после эти данные визуализируем с помощью графиков сделанных в коде на языке python.

Стенд WB-demo-kit v.2:

Часть 1. Листинг кода

Листинг 5 — Код на языке python

```
import paho.mqtt.client as mqtt
import json
from datetime import datetime
from time import sleep
import csv
import os

# Параметры подключения к MQTT-брокеру
HOST = "192.168.1.17"
PORT = 1883
KEEPALIVE = 17

# Словарь с топиками и собираемыми из них параметрами
SUB_TOPICS = {
    '/devices/wb-msw-v3_21/controls/Current Motion': 'motion',
    '/devices/wb-ms_11/controls/Temperature': 'temperature',
    '/devices/power_status/controls/Vin': 'volte'
}
```

```
CSV_FILE = 'data.csv'

CSV_HEADER = ['timestamp', 'motion', 'temperature', 'volte', 'case_no']

def on_connect(client, userdata, flags, rc, properties):

    print("Подключение установлено с кодом результата: " + str(rc))

    for topic in SUB_TOPICS.keys():

        client.subscribe(topic)

        print("Подписан на топик: " + topic)

def on_message(client, userdata, msg):

    payload = msg.payload.decode()

    topic = msg.topic

    param_name = SUB_TOPICS[topic]

    current_time = datetime.now()

    timestamp = current_time.strftime('%Y-%m-%d %H:%M:%S')

    # Инициализация строки с пустыми значениями
    row_data = {

        'timestamp': timestamp,

        'motion': "",

        'temperature': "",
```

```
'volte': ",  
  
    'case_no': HOST[-2:] # Берем последние 2 символа IP-адреса как номер  
кейса  
  
}  
  
# Заполняем только тот параметр, который пришел  
row_data[param_name] = payload  
  
# Проверяем существование файла  
file_exists = os.path.isfile(CSV_FILE)  
  
# Записываем данные в CSV  
with open(CSV_FILE, 'a', newline="") as csvfile:  
    writer = csv.DictWriter(csvfile, fieldnames=CSV_HEADER)  
  
    if not file_exists:  
        writer.writeheader()  
  
    writer.writerow(row_data)  
  
print("%s - %s: %s" % (timestamp, topic, payload))  
  
def main():  
    print("Запуск сбора данных на 10 минут...")  
    print("Данные будут сохранены в файл: " + CSV_FILE)
```

```
print("Собираемые параметры:")
for topic, param in SUB_TOPICS.items():
    print(f" - {param}: {topic}")

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message
client.connect(HOST, PORT, KEEPALIVE)

client.loop_start()

try:
    # Сбор данных в течение 10 минут (600 секунд)
    sleep(600)
except KeyboardInterrupt:
    print("Сбор данных прерван пользователем")

client.loop_stop()
print("Сбор данных завершен. Файл сохранен: " + CSV_FILE)

if __name__ == "__main__":
    main()
```

Часть 2 — Визуализация данных формата csv

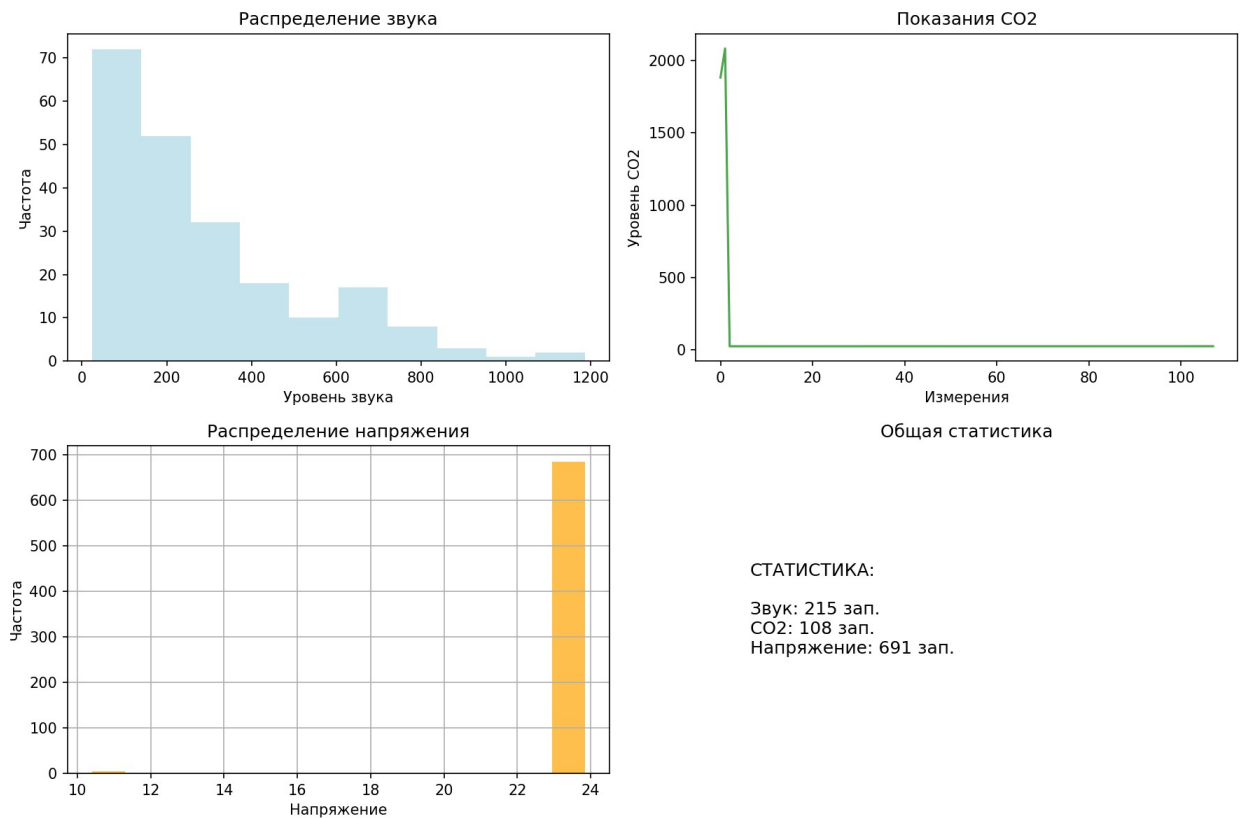


Рисунок 19 — Диаграмма данных

Стенд WB-demo-kit v.3:

Часть 1. Листинг кода

Листинг 6— Код на языке python

```
import paho.mqtt.client as mqtt
import json
from datetime import datetime
import threading

# Параметры подключения к MQTT-брокеру
HOST = "192.168.1.22" # IP чемодана
PORT = 1883 # Стандартный порт подключения для Mosquitto
KEEPALIVE = 60 # Время ожидания доставки сообщения
```

```
# Словарь с топиками и собираемыми из них параметрами
SUB_TOPICS = {
    '/devices/wb-msw-v3_64/controls/Temperature': 'Temperature',
    '/devices/wb-msw-v3_64/controls/CO2': 'CO2',
    '/devices/battery/controls/Voltage': 'Voltage'
}

JSON_LIST = []

# Создание словаря для хранения данных JSON
JSON_DICT = {}
for value in SUB_TOPICS.values():
    JSON_DICT[value] = 0

def on_connect(client, userdata, flags, rc):
    """ Функция, вызываемая при подключении к брокеру """
    print("Connected with result code " + str(rc))

    # Подключение ко всем заданным выше топикам
    for topic in SUB_TOPICS.keys():
        client.subscribe(topic)
```



```

def on_message(client, userdata, msg):

    """ Функция, вызываемая при получении сообщения от брокера """

    payload = msg.payload.decode() # Основное значение

    topic = msg.topic # Топик, из которого пришло сообщение


    param_name = SUB_TOPICS[topic]

    JSON_DICT[param_name] = payload

    JSON_DICT['time'] = str(datetime.now())


    print(topic + " " + payload)


def save_data_periodically():

    """ Сохраняет данные в файл каждые 5 секунд """

    JSON_LIST.append(JSON_DICT.copy())

    with open('data.json', 'w') as file:

        json_string = json.dumps(JSON_LIST)

        file.write(json_string)

    # Запланировать следующий вызов через 5 секунд

    timer = threading.Timer(5.0, save_data_periodically)

    timer.daemon = True

    timer.start()


def main():

    # Создание и настройка экземпляра класса Client для подключения к
    Mosquitto

```

```

client = mqtt.Client()

client.on_connect = on_connect

client.on_message = on_message

client.connect(HOST, PORT, KEEPALIVE)

# Запуск периодической записи данных

save_data_periodically()

client.loop_forever() # Бесконечный внутренний цикл клиента в ожидании
сообщений

if __name__ == "__main__":

    main()

```

Часть 2 — Визуализация данных формата csv

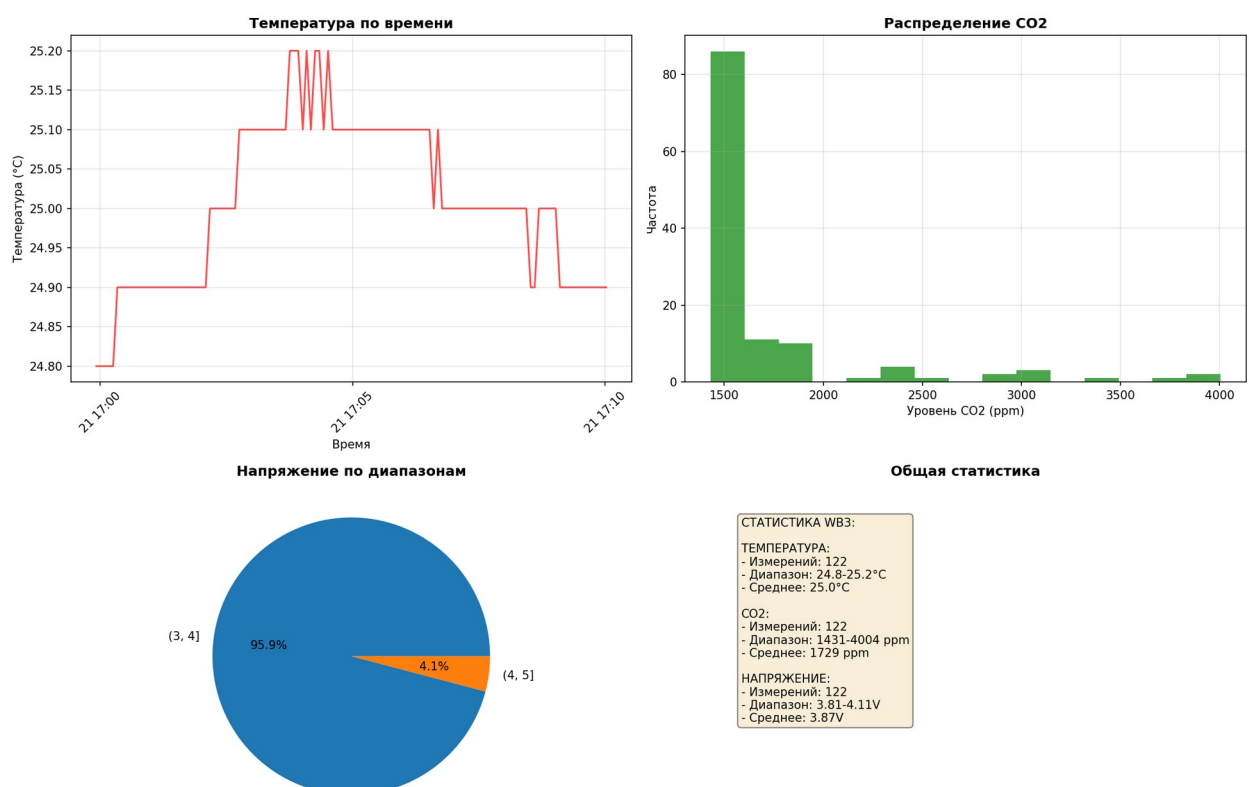


Рисунок 20 — Диаграмма данных

Вывод о проделанных работ

В течение 4 практик мы ознакомились с устройствами в составе стенда и их характеристиками. Описали датчики и исполнительные устройства. Узнали о протоколе MQTT и о том, как с помощью его, подключать топики устройств. Немаловажным было отображение данных с устройств/датчиков, а также их визуализация в диаграммы и графики.

Дополнительная литература

1. Антти, С. Интернет вещей: видео, аудио, коммутация / С. Антти. — Москва : ДМК Пресс, 2019. — 120 с. — ISBN 978-5-97060-761-9. — Текст : электронный // Лань: электронно-библиотечная система. — URL:<https://e.lanbook.com/book/123717> (дата обращения: 25.08.2023). — Режим доступа: для авториз. Пользователей.
2. Гофман, П. М. Промышленный интернет вещей. Компоненты полевого уровня : учебное пособие / П. М. Гофман, П. А. Кузнецов. — Красноярск : СибГУ им. академика М. Ф. Решетнёва, 2022. — 176 с. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/330155> (дата обращения: 25.08.2023). — Режим доступа: для авториз. пользователей.
3. Грингард, С. Интернет вещей: Будущее уже здесь / С. Грингард ; перевод М. Трощенко. — Москва : Альпина Паблишер, 2016. — 188 с. — ISBN 978-5-9614-5853-4. — Текст: электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/87981> (дата обращения: 25.08.2023). — Режим доступа: для авториз. пользователей.